TryNex Lifestyle

Premium Custom Apparel — Bangladesh

Product & Engineering Overview

2026-05-31

1. Product Summary

TryNex Lifestyle is a full-stack custom-apparel e-commerce platform built for the Bangladesh market. Customers can browse premium T-shirts, hoodies, mugs, caps, and gift hampers, design their own products in a 3D-powered Design Studio, and place orders with cash-on-delivery, bKash, Nagad, Rocket, Upay, or card payments. The site is bilingual-aware (English + Bengali keywords) and ships to all 64 districts.

Key Customer Features

- Catalog with category/price/rating filters and instant search
- Product detail with reviews, size/color variants, AR-style 3D preview
- Design Studio with image upload, AI background-remove, HD upscale, layers, fonts, presets
- 3D realtime preview using react-three-fiber + GLB shirt mesh + canvas-composed UV textures
- Cart with 3D viewer of the saved design exactly as composed in studio
- Checkout with split shipping (Dhaka/outside), promo codes, COD 15% advance handling
- Guest checkout that auto-creates a user account and emails credentials
- Order tracking by phone + order number, real-time status updates for admins
- Blog system with categories, related posts, FAQ-aware structured data, share buttons

Admin Features

- Live orders dashboard (auto-refreshes every 3s, BroadcastChannel cross-tab sync)
- Order detail with payment-status workflow (pending !’ submitted !’ verified)
- Bulk and per-design original-asset download (15-min presigned URLs)
- Product, category, blog, hero, testimonial, promo-code, and site-settings CRUD
- Customer list with order history, UTM attribution capture per order

2. Architecture

The repo is a pnpm monorepo with three artifacts: the storefront (React 19 + Vite), the API server (Express 5), and a mockup-sandbox used for component prototyping. PostgreSQL with Drizzle ORM holds all data. Object storage uses an auto-detecting adapter (Cloudflare R2 / AWS S3 / local sidecar) so the platform is fully Replit-independent in production.

Stack

- Frontend: React 19, Vite, Tailwind CSS, framer-motion, react-three-fiber, @tanstack/react-query, Wouter routing, vite-plugin-pwa
- Backend: Node 20, Express 5, Drizzle ORM, PostgreSQL, Pino structured logging, helmet, express-rate-limit, JWT admin sessions
- Storage: Cloudflare R2 (production) via @aws-sdk/client-s3 with auto-fallback to local sidecar in dev
- Auth: bcrypt password hash, Google + Facebook OAuth, guest auto-account creation

• Build/Deploy: Render web service (API) + Cloudflare Pages (storefront), GitHub-connected auto-deploy

Repository layout

• artifacts/trynex-storefront — customer-facing SPA, served as static from Cloudflare Pages
• artifacts/api-server — REST + JSON API, Drizzle migrations, sitemap/robots routes
• packages/api-spec — OpenAPI definition that codegens shared TS client
• packages/api-client-react — generated React Query hooks shared by storefront
• scripts/ — operational scripts (PDF gen, deploy webhooks)

3. Security & Production Hardening

• Helmet headers (frame-ancestors, X-Content-Type-Options, HSTS in production)
• Per-route rate limits: admin login (8/15m), auth (20/15m), orders (30/15m), tracking (20/5m), public reads (200/5m), promo (30/5m)
• Strict CORS allowlist via ALLOWED_ORIGINS — production refuses to start if unset
• Admin JWT secret must be 32+ chars in production, validated at boot
• Order tracking limiter prevents enumeration of TN-XXXXX numbers
• All admin write endpoints behind verifyAdminToken middleware
• DOMPurify sanitization of all user-rendered HTML (blog content)
• Presigned URLs (15 min TTL) for private original-design downloads

4. SEO & Discoverability

• Static SEO meta on every route via SEOHead component (title, description, canonical, OG, Twitter)
• Organization, WebSite SearchAction, ClothingStore (LocalBusiness), Product, BlogPosting, BreadcrumbList, FAQPage JSON-LD
• Bilingual keyword strategy (English + Bengali: ™Y¾›‰Í™ù® ™y¿š¹Ÿ šÉ¾˜)²›é¦œey¶)
• Generated sitemap.xml + robots.txt served from API, mirrored at storefront root
• Image-aware sitemap entries for product detail pages
• Bengali-aware canonical = https://trynexshop.com
• Performance: preconnect to fonts/API, preload hero image, fetchpriority=high on LCP
• PWA manifest + service-worker offline page

5. Order & Payment Flow

A customer places an order; the API creates the order with a TN-XXXXX number, computes shipping (free over Ÿ3 S for Dhaka, configurable by district), applies promo code, and stores UTM attribution. For COD a 15% advance is requested; the customer chooses bKash/Nagad/etc., enters a transaction ID, and the admin verifies in the dashboard. Order status flows: pending !’ processing !’ shipped !’ ongoing !’ delivered, with cancellation supported at any pre-shipped stage. All status changes broadcast to other admin tabs via BroadcastChannel for sub-second cross-device sync.

6. Design Studio & 3D

The Design Studio exposes a 1000×1000 SVG canvas where users place image and text layers within product-specific print zones. composer.ts re-renders these layers onto a 2D canvas (with destination-in alpha-masked color tinting so transparent corners never bleed onto cards). The same composer feeds the realtime react-three-fiber preview: tshirts use a real GLB mesh with UV-mapped overlay; mugs use

a wide cylinder wrap; hoodies/caps/longsleeves use a curved silhouette panel. The cart 3D viewer reuses the composed texture so what the user saw in studio is exactly what they see in cart.

7. Operations

- Hosting: Render web service (Express API) + Cloudflare Pages (storefront, static build)
- DNS: trynexshop.com (root + www !’ Render)
- Object storage: Cloudflare R2 bucket via S3-compatible API
- Database: managed Postgres on Render or external provider, connection via DATABASE_URL
- Logs: structured JSON via pino, viewable in Render dashboard
- Deploy: git push main !’ GitHub !’ Render auto-builds both services
- Monitoring: /api/healthz endpoint for uptime checks

Required production env vars

- DATABASE_URL — Postgres connection string (any standard provider)
- ADMIN_JWT_SECRET — 32+ chars; must differ from JWT_SECRET
- JWT_SECRET — customer JWT signing secret; required in production
- ALLOWED_ORIGINS — comma-separated CORS allowlist (https://trynexshop.com)
- ADMIN_PASSWORD — initial admin password
- R2_ACCOUNT_ID, R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET — Cloudflare R2 (required for storage)
- R2_PUBLIC_BASE_URL — optional: R2 public CDN URL for direct image serving
- GOOGLE_CLIENT_ID / FACEBOOK_APP_ID — optional social login

8. Manual Onboarding Checklist

- Verify domain in Google Search Console + Bing Webmaster Tools, submit sitemap.xml
- Configure R2 bucket + CORS for image hosting
- Set DNS A/CNAME records to Render endpoints
- Set up Render auto-deploy webhook for both services
- Add Meta Pixel / Google Analytics IDs in Site Settings (admin)
- Seed initial categories, products, hero images, and blog posts

9. Replit Independence

Production is fully Replit-independent. No component in the live request path depends on Replit infrastructure.

Production topology

- Storefront: Cloudflare Pages (static build, no server-side Replit dependency)
- API: Render Web Service — Express 5, standard Node 20
- Database: any standard PostgreSQL via DATABASE_URL (Render, Supabase, Neon, etc.)
- Object storage: Cloudflare R2 via S3-compatible API (R2_* env vars)

Dev vs production storage

The Replit Object Storage sidecar (GCS at http://127.0.0.1:1106) is the lowest-priority storage fallback and is only activated when neither R2 nor S3 env vars are set — i.e., only inside the Replit dev sandbox. The API server now hard-exits at boot when NODE_ENV=production and the storage backend resolves to 'replit', preventing silent failures on Render.

Boot-time production guards

- Hard-exit if storage backend = 'replit' (Replit sidecar unavailable on Render)
- Hard-exit if any required env var is absent (DATABASE_URL, ADMIN_JWT_SECRET, JWT_SECRET, ADMIN_PASSWORD, ALLOWED_ORIGINS)
- Logs active storage backend at startup: {"backend":"r2"} visible in Render dashboard
- Warns on absent optional vars (GOOGLE_CLIENT_ID, FACEBOOK_APP_ID, R2_PUBLIC_BASE_URL)

Full audit report: docs/replit-independence.md